

如何在專案中使用基準

來源：<https://codelabs.developers.google.com/codelabs/use-baseline-in-your-project?hl=zh-tw>

步驟數：7

在本程式碼研究室中，瞭解如何在網頁專案中使用 Baseline。

目錄

1. [簡介](#)
2. [在本機電腦上設定試用版](#)
3. [如何將 Baseline 整合至專案](#)
4. [選取不同的基準目標，觀察程式碼輸出內容的變化](#)
5. [指定下游瀏覽器](#)
6. [Lint 和其他工具](#)
7. [總結](#)

步驟 1 · 約 0 分鐘

1. 簡介

Baseline 是一項計畫，旨在清楚說明哪些網頁功能可互通，且目前可安全使用。隨著 Baseline 工具的進步，您現在可以直接在專案中將 Baseline 做為 **Browserslist 查詢** 使用，讓工具鍊根據您選擇的 **Baseline 目標** 變更程式碼輸出內容。

在本程式碼研究室中，您將瞭解如何在範例專案中使用 Baseline，以及如何設定 Baseline 來選取特定目標。您也可以觀察專案工具鍊的輸出內容，如何根據所選的基準目標而變化。

2. 在本機電腦上設定試用版

注意：這個範例包含使用 [webpack](#) (常見且熱門的套件組合工具) 建構的基本 React 應用程式。不過，您使用的打包工具通常不是在專案中使用 Baseline 的重點，任何會尋找 Browserslist 設定的工具鍊，都可以使用本程式碼研究室中概述的一般方法。

首先，前往慣用的終端機應用程式，複製範例存放區，然後輸入專案目錄：

```
git clone git@github.com:GoogleChromeLabs/baseline-demos.git
cd baseline-demos/tooling/webpack
```

此時，示範應用程式已整合 Baseline，但您會想查看從頭開始的提交：

```
git checkout d3793f25
```

複製存放區後，現在可以啟動試用版。這個專案使用 [nvm](#) 管理 Node 版本。如果您已在全域安裝相當新的 Node 版本，可能不需要完成這個步驟，但如果您使用 [nvm](#)，請執行下列指令：

```
nvm install
nvm use
```

接著，請安裝專案的套件：

```
npm install
```

注意：下一個步驟會在您的本機啟動應用程式，但如果您想查看示範，這完全是選用步驟。整合 Baseline 的過程比實際的試用版更重要。

如要查看試用版，請執行下列指令：

○○○

```
npm start
```

然後前往 <http://localhost:8080>。這個範例本身就是資訊卡清單，可使用頁面頂端的表單欄位進行篩選。應用程式本身會混合使用達到 Baseline 門檻的功能。

完成試用後，隨時可以前往終端機並按下 **Ctrl+C** 鍵，停止執行試用。

3. 如何將 Baseline 整合至專案

這項示範一開始不會指定 Browserslist 設定。Browserslist 是一種精簡的查詢語法，可告知工具鍊必須支援的最低瀏覽器版本。舉例來說，使用 `last 3 years` 查詢會指定廣泛的目標。在本示範中，我們將指定符合 Baseline 目標的 Browserslist 查詢，您可以在工具鍊中使用。[基準目標](#)可以是下列其中之一：

- **變動目標**：隨著新版瀏覽器推出，目標也會隨時間更新：
 - **基準：最近推出**，可讓您瞭解核心瀏覽器組在過去 30 個月內，實作了哪些可互通的功能。
 - **基準廣泛支援**：包括已在核心瀏覽器集中實作 30 個月以上的互通功能。
- **固定目標**，代表特定時間點的瀏覽器版本。這些年份介於 2016 年至目前年份。

注意：「核心瀏覽器組合」是指 Chrome、Android 版 Chrome、Firefox、Android 版 Firefox、Edge、macOS 版 Safari 和 iOS 版 Safari。

首先，我們會為專案選取「移動基準」廣泛可用目標。如要這麼做，請開啟 `package.json` 並新增下列內容：

```
"browserslist": "baseline widely available"
```

步驟 4 · 約 0 分鐘

4. 選取不同的基準目標，觀察程式碼輸出內容的變化

您剛才選取「Baseline Widely available」做為示範專案的目標。接著，請建構專案：

```
npm run build
```

由於專案的 `babel.config.js` 中，`@babel/preset-env` 的 `debug` 選項指定為 `true`，因此會產生許多額外輸出內容。首先，請注意套件組合工具統計資料中的 CSS 和 JavaScript 大小：

```
assets by status 213 KiB [emitted]
  asset js/home.5f3c5480.js 208 KiB [emitted] [immutable] [minimized] (name: home) 2 related assets
  asset css/home.20db50ef.css 3.64 KiB [emitted] [immutable] (name: home) 1 related asset
  asset index.html 564 bytes [emitted]
```

請注意，JavaScript 套件為 **208 KiB**，而 CSS 為 **3.64 KiB**。因為這個專案使用 `core-js` 進行 JavaScript 填補，並使用 `autoprefixer` 為尚未完全互通的 CSS 屬性套用廠商專屬前置字元。`core-js` 和 `autoprefixer` 都會受到所選 Baseline Browserslist 查詢的影響。

輸出內容中另一個值得注意的地方，是 Baseline Widely available 的 Browserslist 查詢如何轉譯為 Browserslist 查詢。在專案中，這看起來會類似於：

```
Using targets: {
  "chrome": "108",
  "edge": "108",
  "firefox": "108",
  "ios": "16",
  "safari": "16"
}
```

請注意建構輸出內容中 `core-js` 插入的 Polyfill：

```
The corejs3 polyfill added the following polyfills:
  es.iterator.constructor { "chrome": "108", "edge": "108", "firefox": "108", "ios": "16", "safari": "16" }
  es.iterator.filter { "chrome": "108", "edge": "108", "firefox": "108", "ios": "16", "safari": "16" }
  es.iterator.map { "chrome": "108", "edge": "108", "firefox": "108", "ios": "16", "safari": "16" }
```

如果變更基準目標，這項輸出內容可能會隨之變更。假設您的應用程式必須支援舊版瀏覽器，因為服務水準協議較為嚴格。如果屬於這種情況，您可能會選擇較保守的目標。在 `package.json` 檔案中，變更 Browserslist 設定，反映下列內容：

```
"browserslist": "baseline 2016"
```

這會選取 Baseline 2016 做為目標，並轉換為 Browserslist 查詢。重新執行建構作業後，您會發現工具鍊輸出內容有以下差異：

```
npm run build
```

首先，請注意在 Bundler 統計資料中，專案的 JavaScript 和 CSS 檔案大小變化：

```
assets by status 237 KiB [emitted]
  asset js/home.b228612d.js 232 KiB [emitted] [immutable] [minimized] (name: home) 2 related assets
  asset css/home.0c3e4fd7.css 3.91 KiB [emitted] [immutable] (name: home) 1 related asset
  asset index.html 564 bytes [emitted]
```

您會發現 JavaScript 套件的大小增加了近 30 KiB。由於 autoprefixer 根據 2016 年的基準目標新增更多供應商前置字元，因此專案的 CSS 只會略為變大。另請注意 Browserslist 查詢的變更：

```
Using targets: {
  "chrome": "53",
  "edge": "14",
  "firefox": "49",
  "ios": "10",
  "safari": "10"
}
```

與「Baseline 廣泛支援」目標相比，這些瀏覽器版本非常舊，甚至比 Chromium 還早推出，因此這裡的 Edge 目標版本是 Chromium 前的版本。

注入的 Polyfill 也會變更，這比選取「Baseline Widely available」做為目標時多出許多：`core-js`

The corejs3 polyfill added the following polyfills:

```
es.array.filter { "edge":"14" }
es.iterator.constructor { "chrome":"53", "edge":"14", "firefox":"49", "ios":"10",
"safari":"10" }
es.iterator.filter { "chrome":"53", "edge":"14", "firefox":"49", "ios":"10", "safari":"10"
}
es.object.to-string { "edge":"14", "firefox":"49" }
es.array.includes { "firefox":"49" }
es.string.includes { "edge":"14" }
es.array.map { "firefox":"49" }
es.iterator.map { "chrome":"53", "edge":"14", "firefox":"49", "ios":"10", "safari":"10" }
es.symbol { "edge":"14", "firefox":"49" }
es.symbol.description { "chrome":"53", "edge":"14", "firefox":"49", "ios":"10",
"safari":"10" }
es.array.iterator { "chrome":"53", "edge":"14", "firefox":"49" }
web.dom-collections.iterator { "chrome":"53", "edge":"14", "firefox":"49", "ios":"10",
"safari":"10" }
es.array.push { "chrome":"53", "edge":"14", "firefox":"49", "ios":"10", "safari":"10" }
es.regexp.to-string { "edge":"14" }
es.array.from { "edge":"14", "firefox":"49" }
es.regexp.exec { "chrome":"53", "edge":"14", "firefox":"49", "ios":"10", "safari":"10" }
es.regexp.test { "edge":"14" }
es.error.cause { "chrome":"53", "edge":"14", "firefox":"49", "ios":"10", "safari":"10" }
```

重點是，基準目標可能會對專案工具鍊轉換應用程式的方式產生重大影響。在這個範例中，應用程式非常基本，React 或應用程式本身都沒有太多尖端開發人員體驗功能。如果是複雜程度高出許多的大型應用程式，結果可能會大相逕庭，甚至可能需要更多填補程式、轉換和其他額外程式碼來源，才能符合您選擇的 Baseline 目標。

5. 指定下游瀏覽器

如要複習，Baseline 的目標核心瀏覽器組合包含下列瀏覽器：

- Chrome
- Android 裝置上的 Chrome
- Firefox
- Android 版 Firefox
- 邊緣
- macOS 上的 Safari
- iOS 版 Safari

不過，您可以指定所謂的「下游瀏覽器」。這類瀏覽器的引擎衍生自核心瀏覽器集中的瀏覽器，最常見的是 Chromium。包括 Opera、Samsung Internet 等瀏覽器。除了核心瀏覽器組合中的瀏覽器，您也可以指定這些瀏覽器，只要在任何有效的 Baseline Browserslist 查詢中加入 `with downstream` 即可：

```
"browserslist": "baseline widely available with downstream"
```

這會根據「廣泛可用的基準」目標，指定下游瀏覽器。如要查看這項設定如何解析為 Browserslist 查詢，請重建專案：

```
npm start
```

然後注意 Browserslist 查詢的變化：

```
Using targets: {
  "android": "108",
  "chrome": "108",
  "edge": "108",
  "firefox": "108",
  "ios": "16",
  "opera": "94",
  "opera_mobile": "80",
  "safari": "16",
  "samsung": "21"
}
```

您也可以依年份指定下游瀏覽器。例如：

```
"browserslist": "baseline 2016 with downstream"
```

完成這項設定後，Browserslist 查詢就會隨之變更：

```
Using targets: {  
  "android": "53",  
  "chrome": "53",  
  "edge": "14",  
  "firefox": "49",  
  "ios": "10",  
  "opera": "40",  
  "opera_mobile": "80",  
  "safari": "10",  
  "samsung": "6.2"  
}
```

6. Lint 和其他工具

Browserslist 內建的 Baseline 查詢可供打包工具和工具鍊的其他部分使用，但其他工具 (例如採用 Baseline 目標做為設定一部分的 Lint 工具) 也很有價值。

以支援 Baseline 的 Linter 為例，[ESLint](#) 是 CSS Linting 的一部分，可使用 [@eslint/css](#) 提供 `use-baseline` 規則，讓您指定 Baseline 新推出、Baseline 廣泛支援或 Baseline 年份。社群 [@html-eslint/eslint-plugin](#) 套件中也有類似規則，可讓您對 `eslint.config.js` 檔案中的 HTML 功能執行相同操作：

```
export default [  
  /* Omitted JS linting rules ... */  
  // Lint CSS files for Baseline:  
  {  
    files: ["**/*.css"],  
    plugins: {  
      css  
    },  
    language: "css/css",  
    rules: {  
      "css/no-duplicate-imports": "error",  
      // Lint CSS files to make sure they are using  
      // only Baseline Widely available features:  
      "css/use-baseline": ["warn", {  
        available: "widely"  
      }]  
    },  
  },  
  // Lint HTML and JSX files for Baseline:  
  {  
    files: ["**/*.html"],  
    ...html.configs["flat/recommended"],  
    rules: {  
      // Lint HTML files to make sure they are using  
      // only Baseline Widely available features:  
      "@html-eslint/use-baseline": ["warn", {  
        available: "widely"  
      }]  
    },  
  }  
];
```

這項設定有幾點需要注意：

1. HTML 和 CSS 檢查套件都使用 `use-baseline` 規則，並透過 `available: "widely"` 設定選項設為「廣泛適用」。
2. 對於這兩個 Linting 套件，違規事項的記錄層級會設為 `"warn"`。這項屬性可以設為 `"error"`，以錯誤代碼退出，避免發生建構作業。

您可能在先前執行 `npm run build` 時看過 Linter 輸出內容，但如要單獨查看 Linter 輸出內容，可以執行下列指令：

```
npm run lint
```

輸出內容會醒目顯示專案 CSS 中的多項警告：

```
/var/www/baseline-demos/tooling/webpack-browserslist-config-baseline/src/css/normalize.css
 222:3  warning  Property 'outline' is not a widely available baseline feature  css/use-
baseline

/var/www/baseline-demos/tooling/webpack-browserslist-config-baseline/src/css/styles.css
 62:3   warning  Property 'outline' is not a widely available baseline feature
        css/use-baseline
 81:23  warning  Value 'subgrid' of property 'grid-template-rows' is not a widely available
baseline feature  css/use-baseline
```

注意：如果偏好使用 Stylelint 進行 CSS Linting，可以 使用 [stylelint-plugin-use-baseline](#) 套件。

7. 總結

在專案中使用 Browserslist 提供的 Baseline 查詢時，您需要瞭解基礎建構工具和 Browserslist 本身，但將查詢放入專案的動作本身很簡潔。使用這項功能的主要優點是，您不必再考慮支援的瀏覽器版本號碼，而是以 Baseline 目標為準，讓系統為您完成繁重的工作。

此外，[這個範例還有一個在 Rollup 上執行的版本](#)，您也可以使用該範例，大致按照這個程式碼研究室的步驟操作。

其他組合工具也開始提供基準支援。舉例來說，Vite 7 以上版本[預設會以 Baseline 廣泛支援的瀏覽器為目標](#)，而這類瀏覽器在幕後使用 Rollup。

無論您決定採用哪種做法，只要在專案中導入 Browserslist 提供的 Baseline 查詢，並[選取適合您的 Baseline 目標](#)，就能以更簡單可靠的方式指定目標瀏覽器。
